Bonus: Missing Data

Malcolm Barrett

Stanford University

- `lm()` and `glm()` drop rows with missing values silently
- Every model we have fit in this course has been a complete-case analysis

Dropping the missing values

- Our own exercises say: “We also have a lot of missingness for `wait_minutes_actual_avg`, so we’ll drop unobserved values for now”

- We dropped the unobserved values, and we postponed the problem

We come back to it here

How much is missing?

```
seven_dwarfs_train_2018 |>
  filter(wait_hour == 9) |>
  summarize(
    days = n(),
    missing_actual = sum(is.na(wait_minutes_actual_avg)),
    prop_missing = mean(is.na(wait_minutes_actual_avg))
  )
```

```
# A tibble: 1 × 3
   days missing_actual prop_missing
  <int>          <int>        <dbl>
1   354            207        0.585
```

At the 9 AM hour, 207 of 354 days are missing the actual wait time; across the full data, 88.7% of actual wait times are missing

A known benchmark

- The malaria data has no missing values, so we will remove some on purpose

- Then we compare each remedy to the answer we already know

The benchmark: the full-data IPW estimate of the effect of net use on malaria risk, -12.5 (from the outcome model module)

The missingness indicator

Two nodes for one variable

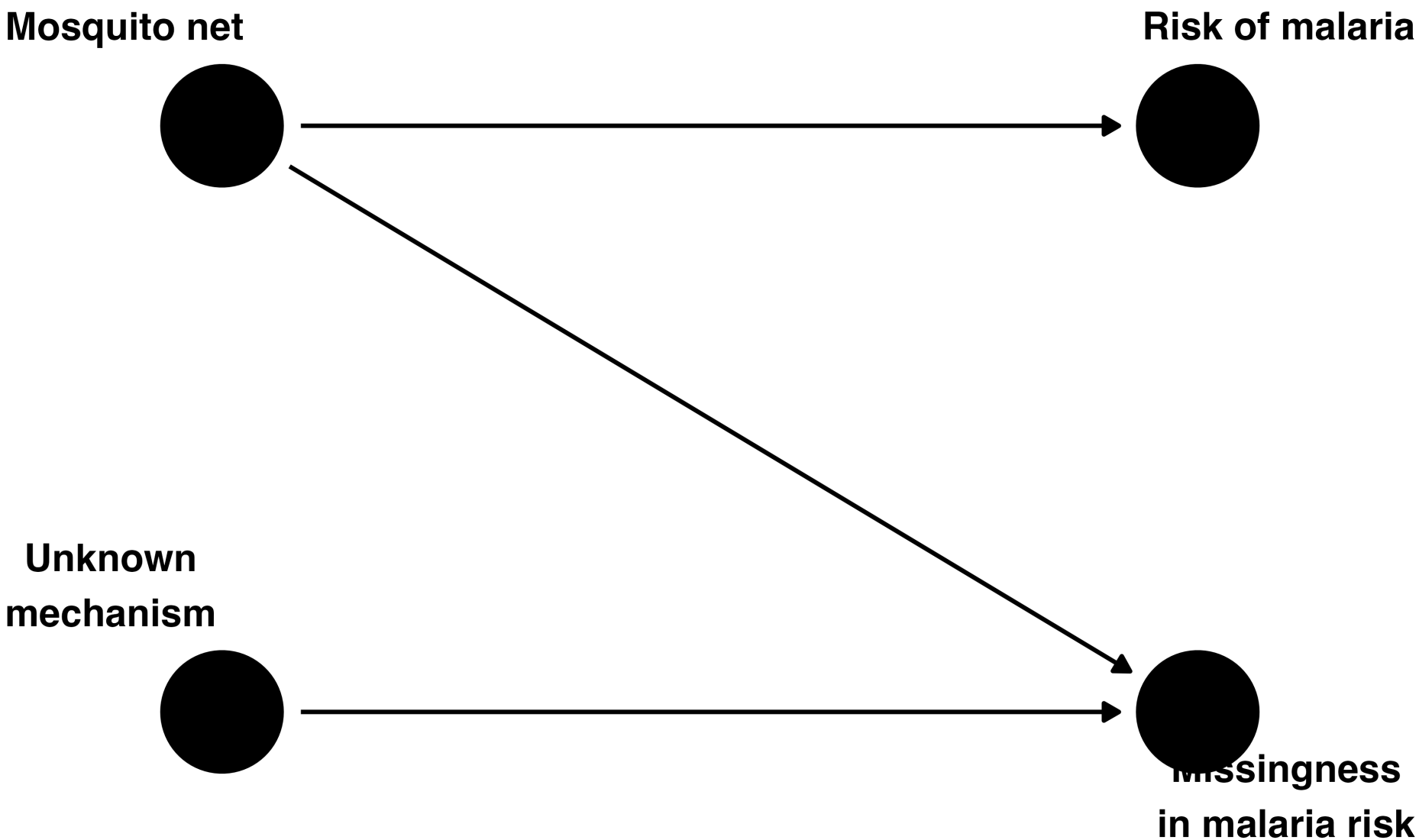
- We draw two nodes for a given variable: one represents the true values, and the other represents a *missingness indicator*, whether we observed the value or not

- We are always conditioning on the data we actually have

Two nodes for one variable

With missingness, that usually means conditioning on *complete* observations: the subset where every variable we need is observed

A DAG with missingness



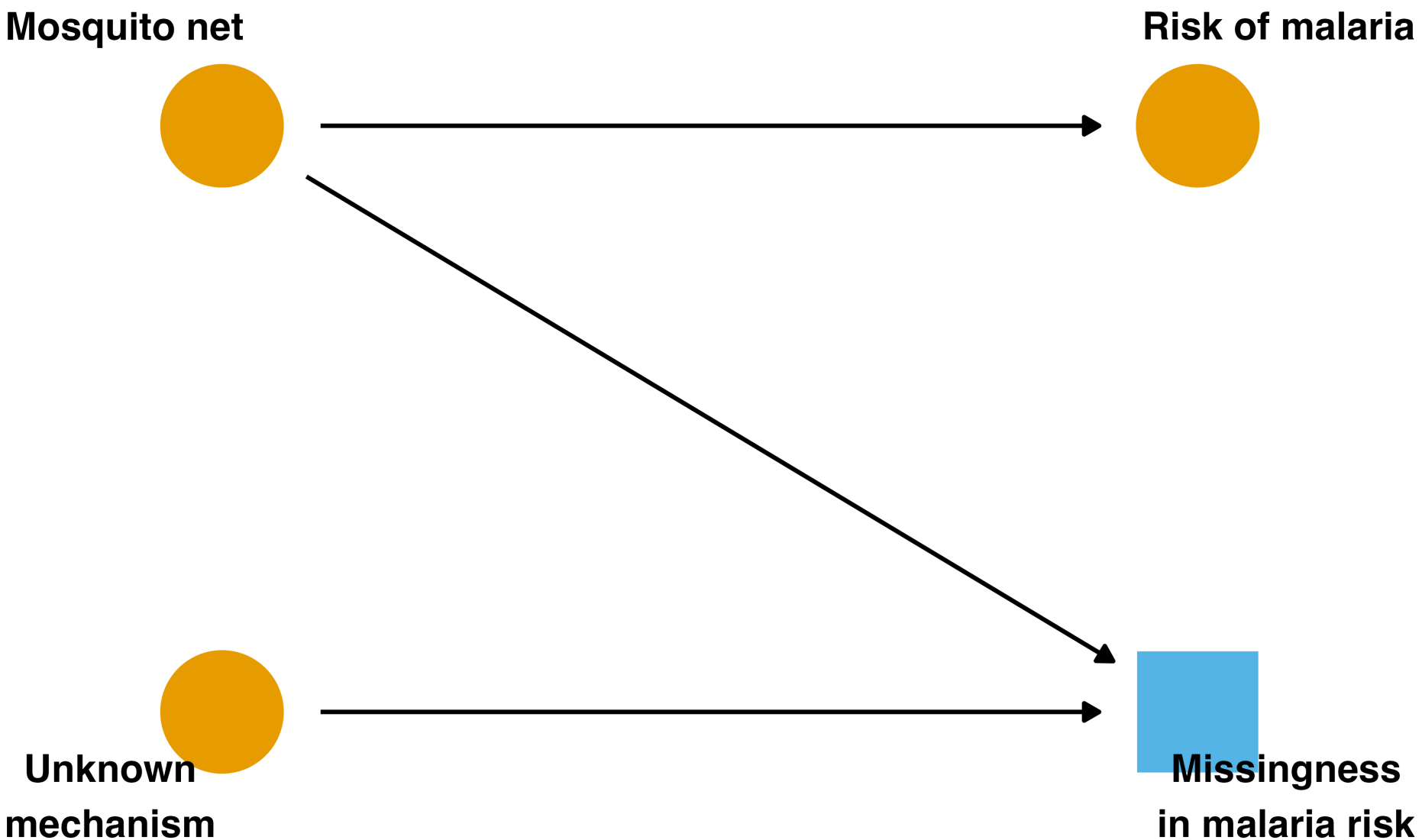
Reading this DAG

- The missingness of malaria risk is related to net use and to an unknown mechanism

- The unknown mechanism is random, but the mechanism related to net use is not, since net use is the exposure

When complete cases are enough

Conditioning on the indicator



A collider that opens no backdoor

- The missingness indicator is a collider, and conditioning on it is exactly what complete-case analysis does

- In this DAG, though, conditioning on it does not open a backdoor path between net use and malaria risk

A collider that opens no backdoor

- It does bias the relationship between the unknown mechanism and net use, but we do not care about that relationship

Recoverable effects

- The causal effect is *recoverable*: we can still estimate it without bias with complete-case analysis

- Because we only use complete observations, we have a smaller sample size and thus reduced precision

We are unable to correctly estimate the *mean* of malaria risk because values are systematically missing by net use

When complete cases are not enough

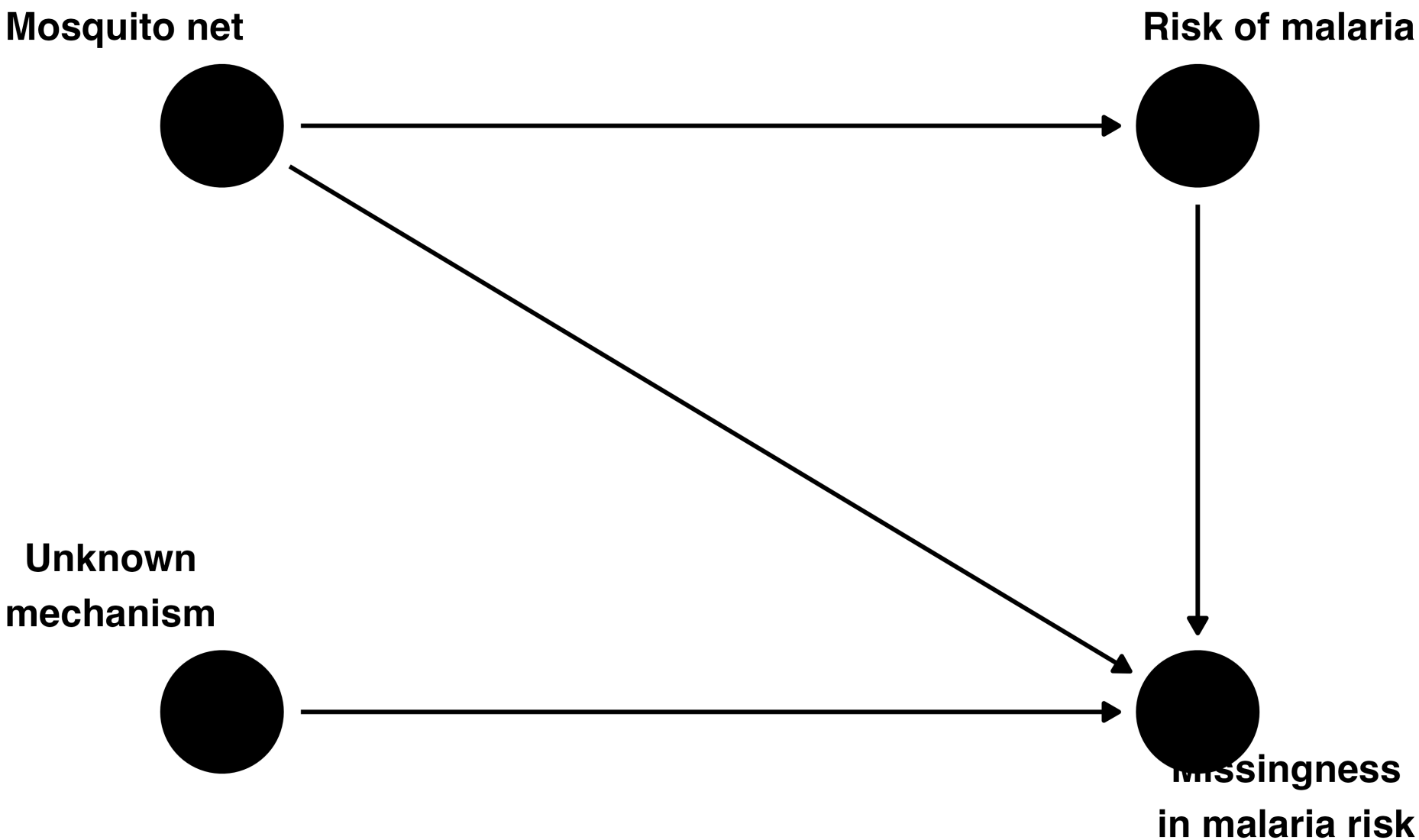
Missingness from the value itself

- Is it feasible that the missingness of malaria risk is related to its own values?

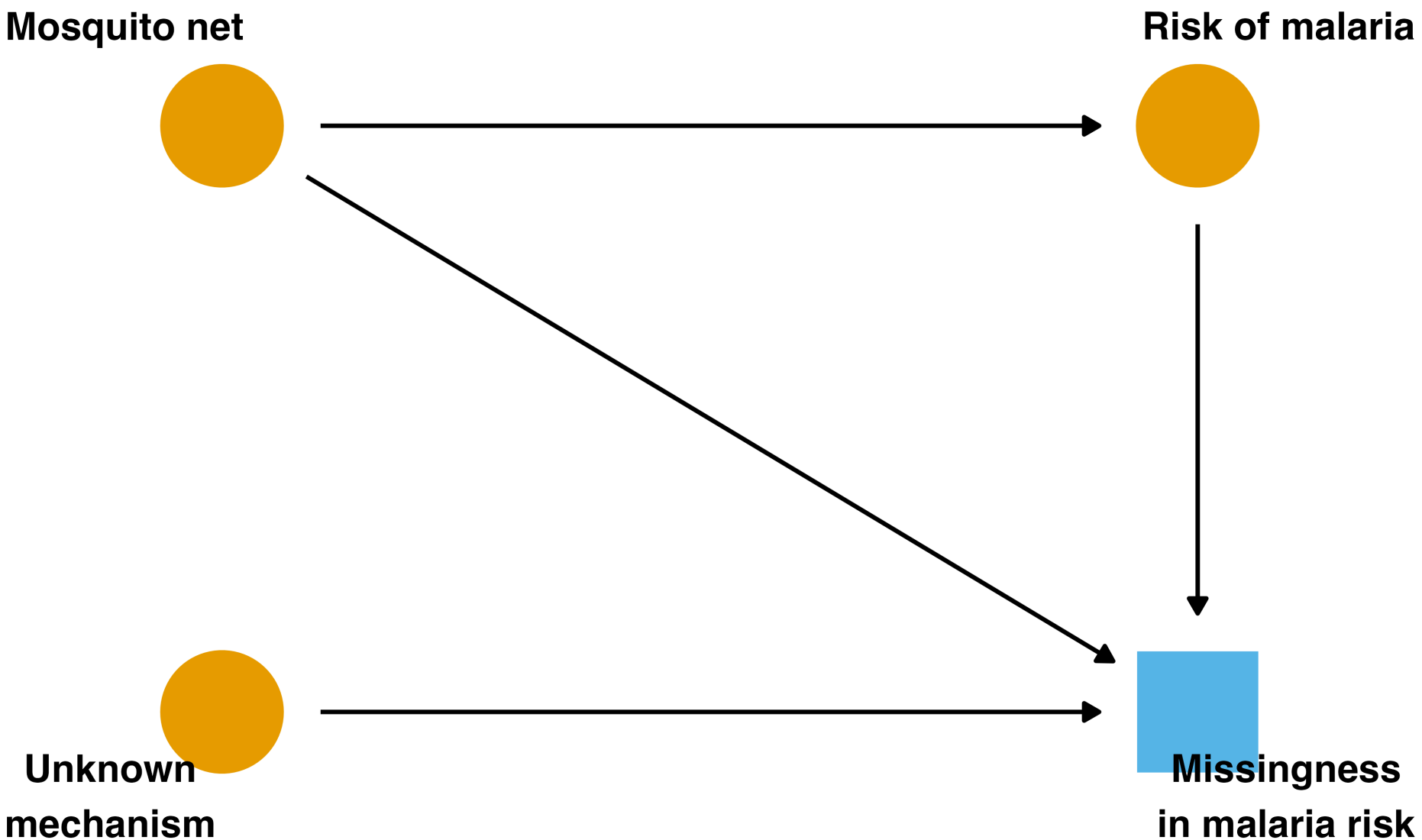
- It could be that a person's malaria risk is so severe that the measurement never happens

The values of malaria risk themselves influence whether we managed to measure them

An arrow from the outcome



Conditioning now opens a backdoor



No way to close it

- Whether malaria risk was measured is now a descendant of both the exposure and the outcome

- Conditioning on the indicator opens a backdoor path, creating nonexchangeability

There is no way to close the backdoor path opened by conditioning on missingness

More than backdoor paths

- What effects we can recover is more complicated than determining backdoor paths alone

More than backdoor paths

- When there are no backdoor paths, we can calculate the causal effect, but we may not be able to calculate other statistics, like the mean of the exposure or outcome

- When conditioning on missingness opens a backdoor path, sometimes we can close it, and sometimes we cannot

Five structures of missingness

Five DAGs

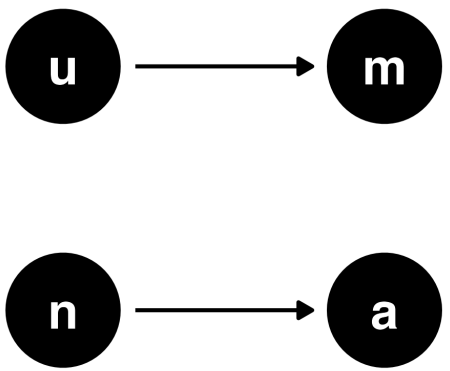
- Consider five DAGs, where `a` is malaria risk, `n` is net use, `u` is unknown, and `m` is missing

- Each represents a simple but differing structure of missingness

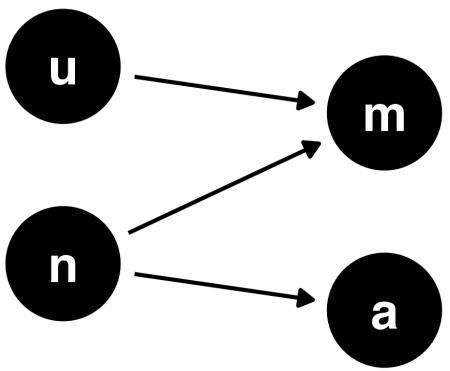
In DAGs 1-3, malaria risk has missingness; in DAGs 4-5, net use does

Five structures

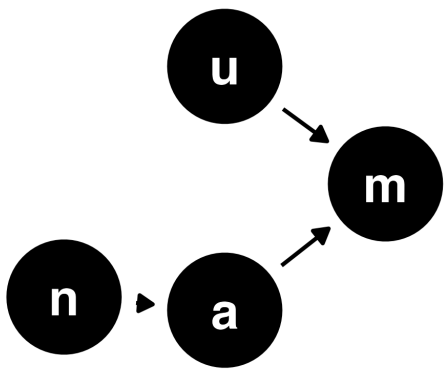
1: malaria risk is missing



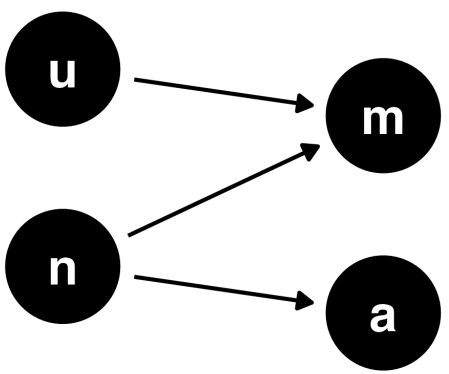
2: malaria risk is missing



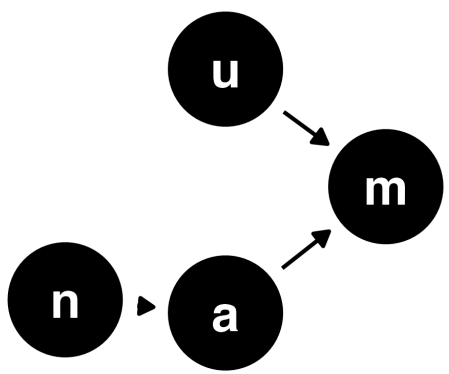
3: malaria risk is missing



4: net use is missing



5: net use is missing

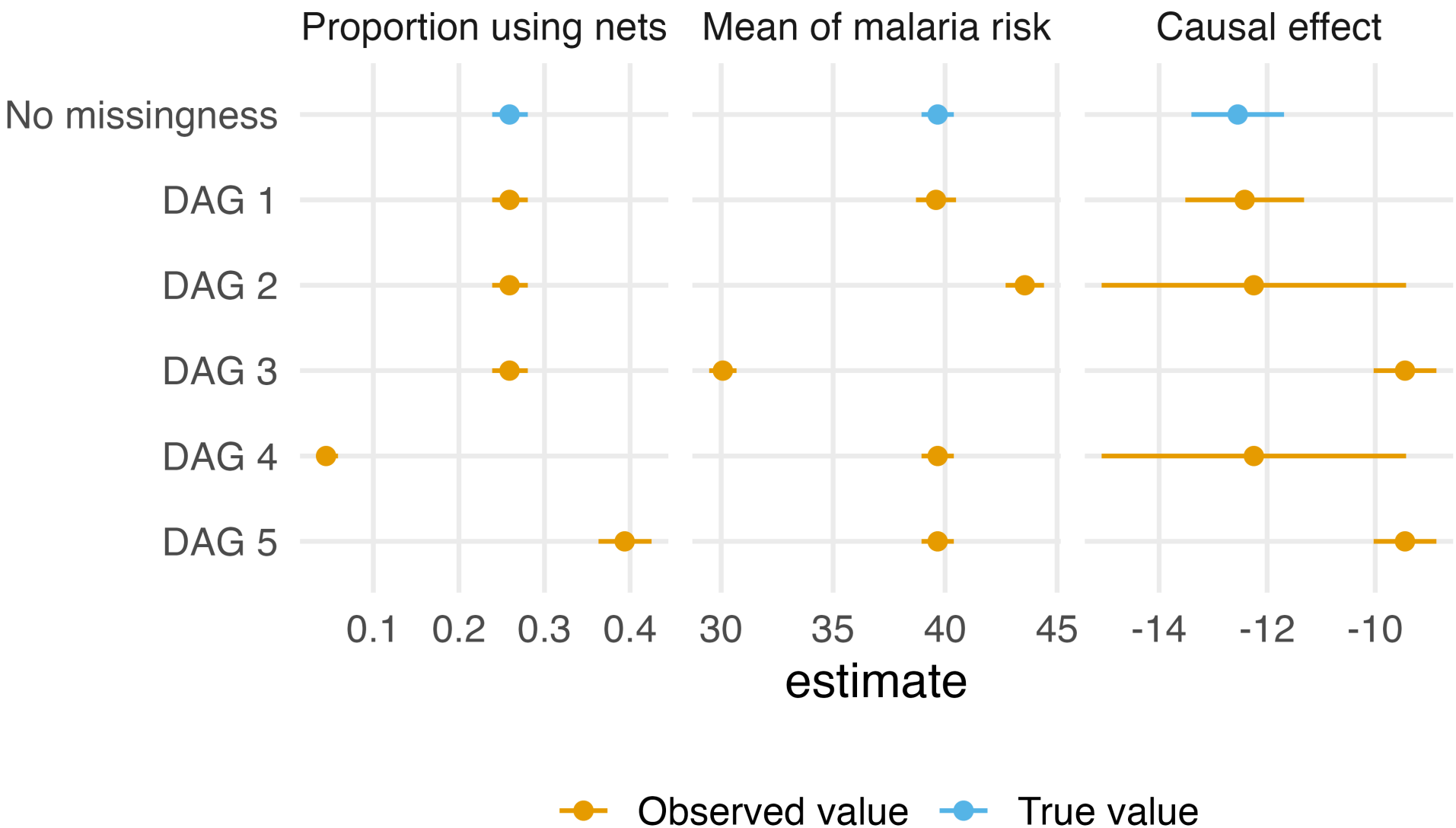


Impose the five structures on the data

```r
set.seed(123)
n <- nrow(net_data)
m1 <- as.logical(rbinom(n, 1, 0.35))
m2 <- as.logical(
  if_else(net_data$net, rbinom(n, 1, 0.9), rbinom(n, 1, 0.15))
)
# the threshold 38 sits near the median of malaria risk
m3 <- as.logical(
  if_else(net_data$malaria_risk > 38, rbinom(n, 1, 0.9), rbinom(n, 1, 0.05))
)
# the same structures, but net use gets the resulting missingness
m4 <- m2
m5 <- m3
```

For each structure, we set the chosen column to missing, then estimate the proportion using nets, the mean of malaria risk, and the IPW effect

What we can estimate



What we can estimate

- With no missingness, we can estimate all three quantities
- DAG 1 loses only precision
- DAG 2 loses the mean of malaria risk

What we can estimate

- DAG 3 loses the mean of malaria risk and the causal effect
- DAG 4 loses the proportion using nets
- DAG 5 loses everything but the mean of malaria risk

What we can estimate

The causal effect is recoverable in structures 1, 2, and 4, and lost in 3 and 5

Further reading

See Moreno-Betancur et al. (2018) for a comprehensive overview of what effects are recoverable from different structures of missingness

Moreno-Betancur M, Lee KJ, Leacy FP, White IR, Simpson JA, Carlin JB (2018). Canonical causal diagrams to guide the treatment of missing data in epidemiologic studies. American Journal of Epidemiology. doi:10.1093/aje/kwy173

MCAR, MAR, and MNAR

Three terms you will see

In the grand tradition of statisticians being bad at naming things, you will also commonly see missingness discussed in terms of ***missing completely at random***, ***missing at random***, and ***missing not at random***

We can explain these ideas by the causal structure of missingness and the availability of variables and values related to that structure in your data

Missing completely at random

- MCAR: there are missing values, but the causes of missingness are such that the missingness process is unrelated to the causal structure of your question

- In other words, the only problem with missingness is a reduction in sample size

This is structure 1, where the only cause of missingness is the unknown mechanism

Missing at random

- MAR: the causes of missingness are related to the causal structure of the research problem, but missingness only depends on the variables and values in the data that we have actually observed

This is structure 2, where the missingness of malaria risk depends on net use, which we always observe

Missing not at random

- MNAR: the causes of missingness are related to the causal structure of the research problem, but the process is related to values we are missing

- A classic example is when a variable's missingness is impacted by itself: higher values of x are more likely to be missing in x, as in structure 3

We do not have that information because, by definition, it is missing

Why we avoid these terms

- These terms do not always tell you what to do next
- We will avoid them in favor of explicitly describing the missingness generation process

Next, we impose a mechanism on the malaria data and try to repair it

Regression imputation

A missingness mechanism

```
p_missing <- plogis(
  -0.8 + 1.4 * net_data$net + 0.3 * as.numeric(scale(net_data$income))
)
set.seed(88)
missing_mar <- rbinom(n, 1, p_missing) == 1

net_data_mar <- net_data |>
  select(net, income, health, temperature, malaria_risk) |>
  mutate(malaria_risk = if_else(missing_mar, NA_real_, malaria_risk))
```

Malaria risk now goes missing more often for net users and for higher-income households: about 40% of the values are missing, for reasons we can see in the data

The IPW pipeline

```
fit_ipw_net <- function(.data) {
  propensity_model <- glm(
    net ~ income + health + temperature,
    family = binomial(),
    data = .data
  )
  weighted_data <- propensity_model |>
    augment(type.predict = "response", data = .data) |>
    mutate(wts = wt_ate(.fitted, net, exposure_type = "binary"))
  ipw_model <- lm(malaria_risk ~ net, data = weighted_data, weights = wts)
  ipw(propensity_model, ipw_model)
}
```

The pipeline from the outcome model module: propensity score model, ATE weights, weighted outcome model, `ipw()`

Complete-case analysis

```
net_data_complete <- net_data_mar |> drop_na()

mar_complete_case <- fit_ipw_net(net_data_complete) |>
  tidy(conf.int = TRUE)

mar_complete_case
```

```
# A tibble: 1 × 7
  term  estimate std.error statistic p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl>   <dbl>    <dbl>
1 diff     -12.8     0.776     -16.4       0    -14.3
# ℹ 1 more variable: conf.high <dbl>
```

-12.8 (95% CI -14.3, -11.2): the effect survives, with a wider interval, as the missingness DAG suggested

Complete-case analysis

- We filtered to the complete cases ourselves before fitting either model

- Had we let `lm()` drop the missing rows silently, `ipw()` would error: the propensity score model would see all 1752 rows and the outcome model only the 1047 complete ones

Complete-case analysis

- Complete-case analysis is an analytic choice, and here we have to make it explicitly

The effect is recoverable here, but the mean of malaria risk is not: the complete cases average 42.6 against the full-data 39.7

Regression imputation

- The analysis variables predict malaria risk well, so we can use them to fill in the values we failed to measure

- Fit a model for malaria risk on the complete cases, then plug its prediction in wherever the value is missing

Regression imputation

- When fitting imputation models, include every variable that appears in your later models, in the same form it takes there

Regression imputation

```
imputation_model <- lm(
  malaria_risk ~ net + income + health + temperature,
  data = net_data_mar
)

net_data_imputed <- imputation_model |>
  augment(newdata = net_data_mar) |>
  mutate(malaria_risk = coalesce(malaria_risk, .fitted))
```

`coalesce()` keeps the observed value where we have one and uses the model's prediction where we do not

The imputed estimate

```
mar_regression <- fit_ipw_net(net_data_imputed) |>
  tidy(conf.int = TRUE)

mar_regression
```

```
# A tibble: 1 × 7
  term  estimate std.error statistic p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl>   <dbl>    <dbl>
1 diff     -12.5     0.341     -36.5       0    -13.1
# ℹ 1 more variable: conf.high <dbl>
```

-12.5 (95% CI -13.1, -11.8): very close to the benchmark

Standard errors after imputation

- Each of the 705 missing values got a single predicted value, and the outcome model then treated it like real data

Standard errors after imputation

- The standard error is 0.34, *narrower* than the 0.44 we had with no missing data at all

- Imputed values cannot make us more certain than measured ones

Standard errors after imputation

- If you use this approach, include the fitting of the imputation model in your bootstrap so the standard errors reflect its uncertainty

Deterministic imputation understates uncertainty

Multiple imputation

Single imputation is inefficient

- Regression imputation relies on a single imputed value for each missing observation

- Done correctly, with the imputation model inside the bootstrap, this plug-in approach is generally inefficient: we end up with wide confidence intervals, reflecting the fact that we relied on a single imputed value

Single imputation is inefficient

Multiple imputation instead draws predicted values from a distribution, capturing the uncertainty inherent in the missing data

Many completed datasets

- We generate several completed datasets whose imputed values differ from draw to draw
- The values we observed are identical in every dataset; the values we imputed vary
- The variation between datasets is the uncertainty from the imputation

Two rules for causal imputation models

1. The imputation model must include *every variable* in the outcome model and the propensity score model: the confounders, the exposure, and the outcome itself

Two rules for causal imputation models

1. Mirror the same functional forms you will use downstream: if a variable gets a spline or an interaction later, it needs one in the imputation model too

A typical causal analysis

1. Impute multiple completed datasets with a model that includes the outcome and all covariates
2. Estimate the treatment effect within each dataset: propensity score model, weights, weighted outcome model

A typical causal analysis

1. Pool the estimates with Rubin's rules to get one effect and valid standard errors

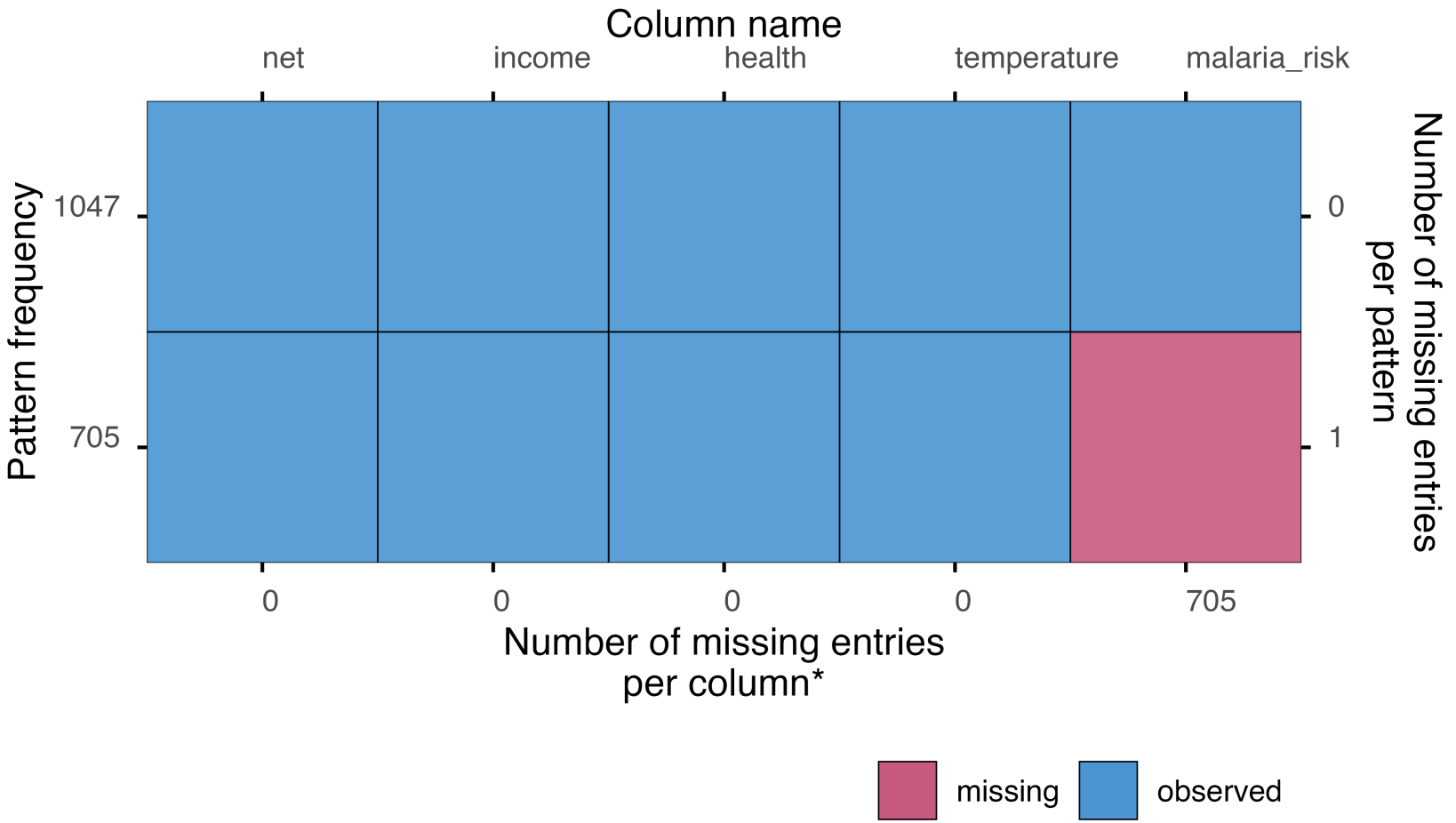
Rule of thumb: the number of imputations should roughly match the percentage of incomplete cases

The missingness pattern

`plot_pattern()` shows which variables are missing and in what combinations: here, one block of 705 rows missing malaria risk

```
1 plot_pattern(net_data_mar)
```

The missingness pattern



Interactions in the imputation model

- In the malaria data, the effect of net use on malaria risk varies with income

- An imputation model with only main effects would fill in values from an effect that is too weak, and the pooled estimate would inherit that bias

Interactions in the imputation model

- Rule 2 again: the marginal effect averages over that heterogeneity, so the imputation model has to represent it

Build the imputation frame

```
net_data_to_impute <- net_data_mar |>
  mutate(
    net_income = as.numeric(net) * income,
    net_health = as.numeric(net) * health,
    net_temperature = as.numeric(net) * temperature
  )
```

The product columns are complete, so `mice()` never imputes them; they serve as predictors in the imputation model for malaria risk

Impute with mice()

```
imp <- mice(
  net_data_to_impute,
  m = 40,
  method = "pmm",
  seed = 20260814,
  print = FALSE
)
```

Impute with mice()

- Predictive mean matching fills each missing value with an observed value borrowed from a case whose prediction is close

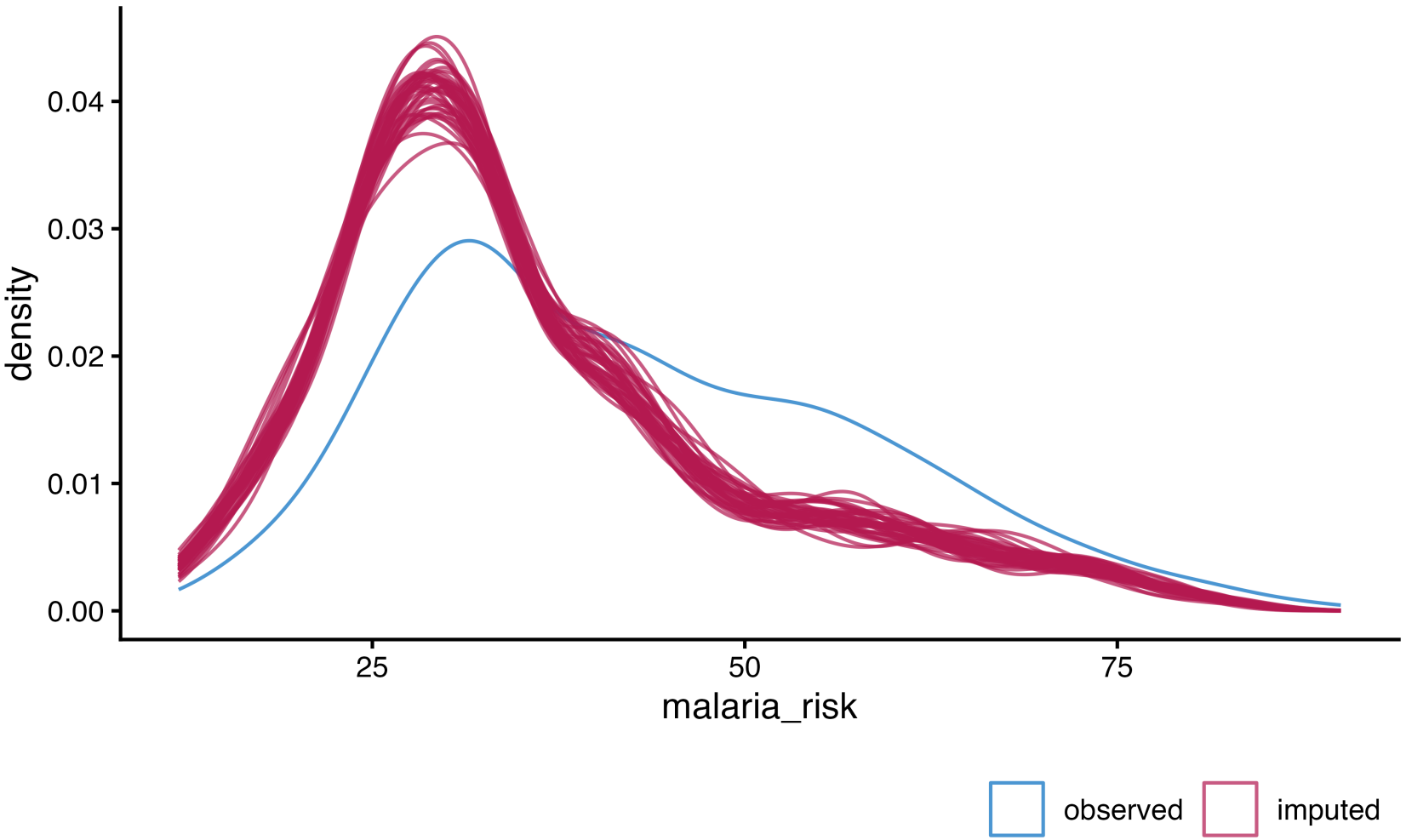
- About 40% of the cases are incomplete, so the rule of thumb says about 40 imputations; we use `m = 40`

Observed versus imputed

The imputed values should be plausible, and their distribution should reflect the mechanism

```
ggmice(imp, aes(x = malaria_risk, group = .imp)) +
  geom_density()
```

Observed versus imputed



Reading the densities

- The observed values (blue) center higher than the imputed values (red)

- That is what the mechanism implies: net users are missing malaria risk more often, and net users have lower risk

Reading the densities

- The imputation model can see this because net use is one of its predictors

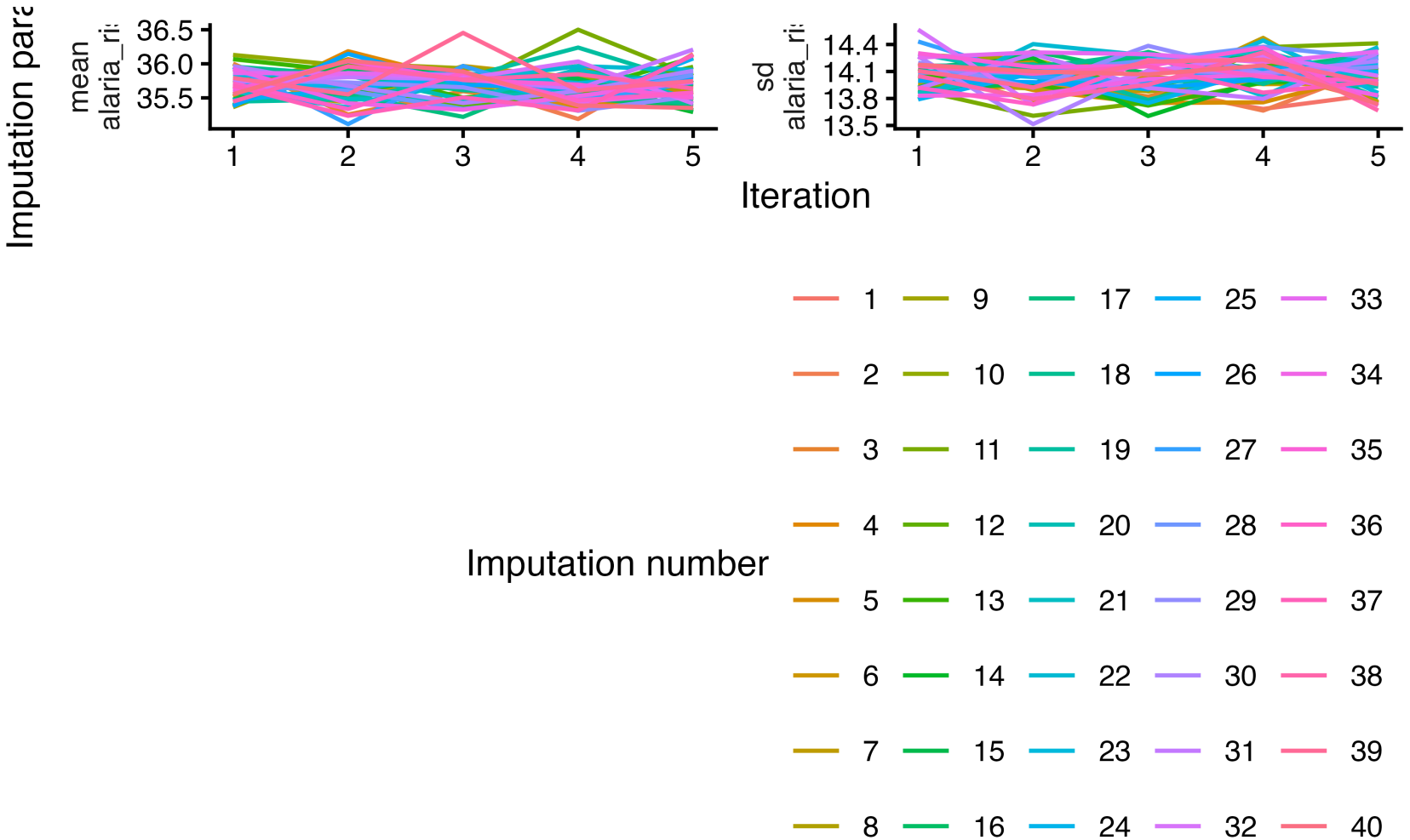
Did the algorithm converge?

- `mice()` imputes each variable with missingness iteratively, conditional on the others

- The trace plot shows the mean and spread of the imputed values across iterations; look for well-mixed lines with no trend

```
plot_trace(imp, "malaria_risk")
```

Did the algorithm converge?



Fit within each imputation

```
fits <- with(imp, {
  ps <- glm(net ~ income + health + temperature, family = binomial())
  w <- wt_ate(fitted(ps), net, exposure_type = "binary")
  om <- lm(malaria_risk ~ net, weights = w)
  ipw(ps, om)
})
```

`with()` runs the block once per completed dataset: forty propensity score models, forty sets of weights, forty outcome models, forty `ipw()` results

Refit the propensity model every time

- The whole analysis is rebuilt inside each imputation, including the propensity score model

- Averaging the propensity scores across imputations and weighting with one averaged set hides the propensity model's estimation uncertainty from the pooling step

Refit the propensity model every time

- Here only the outcome is imputed, so each refit returns the same model; when confounders or the exposure have missingness, the refits differ, and that variation belongs in the pooled standard error

The propensity score model is part of the analysis, so it belongs inside the imputation loop

Rubin's rules

```
estimates <- map_dfr(fits$analyses, tidy)

estimates |>
  summarize(
    pooled = mean(estimate),
    within_var = mean(std.error^2),
    between_var = var(estimate),
    pooled_se = sqrt(within_var + (1 + 1 / n()) * between_var)
  )
```

```
# A tibble: 1 × 4
  pooled within_var between_var pooled_se
   <dbl>      <dbl>       <dbl>     <dbl>
1  -12.1      0.163      0.0798     0.495
```

The pooled estimate is the average across imputations; its variance is the average within-imputation variance plus the between-imputation variance, inflated by `1 + 1/m`

Pooling with pool_ipw()

```
mar_mi <- pool_ipw(fits) |>
  tidy(conf.int = TRUE)

mar_mi
```

```
# A tibble: 1 × 8
  term  estimate std.error statistic    df  p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl> <dbl>    <dbl>    <dbl>
1 diff     -12.1    0.495     -24.4  269. 4.96e-70    -13.0
# ℹ 1 more variable: conf.high <dbl>
```

-12.1 (95% CI -13.0, -11.1): the interval covers the benchmark, and the standard error accounts for the imputation

Pooling with pool_ipw()

- The effect labels survive: the pooled result still knows it is a marginal difference

- The complete-data degrees of freedom are read from the outcome models, with a small-sample adjustment

Pooling with pool_ipw()

- `mice::pool()` also works with these results

- Results from balancing weights pool with the same verb

What imputation cannot fix

Missingness from the value itself

```r
p_missing <- plogis(-0.5 + 3 * as.numeric(scale(net_data$malaria_risk)))
set.seed(789)
missing_mnar <- rbinom(n, 1, p_missing) == 1

net_data_mnar <- net_data |>
  select(net, income, health, temperature, malaria_risk) |>
  mutate(malaria_risk = if_else(missing_mnar, NA_real_, malaria_risk))
```

This is structure 3: the worse the malaria risk, the less likely we measured it, and about 41% of the values are gone

Complete-case analysis

```
mnar_complete_case <- fit_ipw_net(drop_na(net_data_mnar)) |>
  tidy(conf.int = TRUE)

mnar_complete_case
```

```
# A tibble: 1 × 7
  term  estimate std.error statistic p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl>   <dbl>    <dbl>
1 diff     -9.40     0.246     -38.2       0    -9.88
# ℹ 1 more variable: conf.high <dbl>
```

-9.4 (95% CI -9.9, -8.9): far from the benchmark, with an interval that excludes it entirely

The same analysis

```
imp_mnar <- net_data_mnar |>
  mutate(
    net_income = as.numeric(net) * income,
    net_health = as.numeric(net) * health,
    net_temperature = as.numeric(net) * temperature
  ) |>
  mice(m = 40, method = "pmm", seed = 20260814, print = FALSE)
```

Same frame, same method, same number of imputations, same seed

The same analysis

```
fits_mnar <- with(imp_mnar, {
  ps <- glm(net ~ income + health + temperature, family = binomial())
  w <- wt_ate(fitted(ps), net, exposure_type = "binary")
  om <- lm(malaria_risk ~ net, weights = w)
  ipw(ps, om)
})
```

The same analysis

```
mnar_mi <- pool_ipw(fits_mnar) |>
  tidy(conf.int = TRUE)

mnar_mi
```

```
# A tibble: 1 × 8
  term  estimate std.error statistic    df  p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl> <dbl>    <dbl>    <dbl>
1 diff     -10.7     0.330     -32.4  266. 2.79e-94    -11.3
# ℹ 1 more variable: conf.high <dbl>
```

-10.7 (95% CI -11.3, -10.0): closer to the benchmark than the complete cases, but the interval still excludes it

Why imputation cannot help

- When missingness depends on the missing values themselves, conditioning on the indicator opens a backdoor path that nothing observed can close

- The imputation model only sees the observed values, and under this mechanism they are a truncated sample

Why imputation cannot help

- Multiple imputation moves the estimate toward the benchmark, but it does not reach it

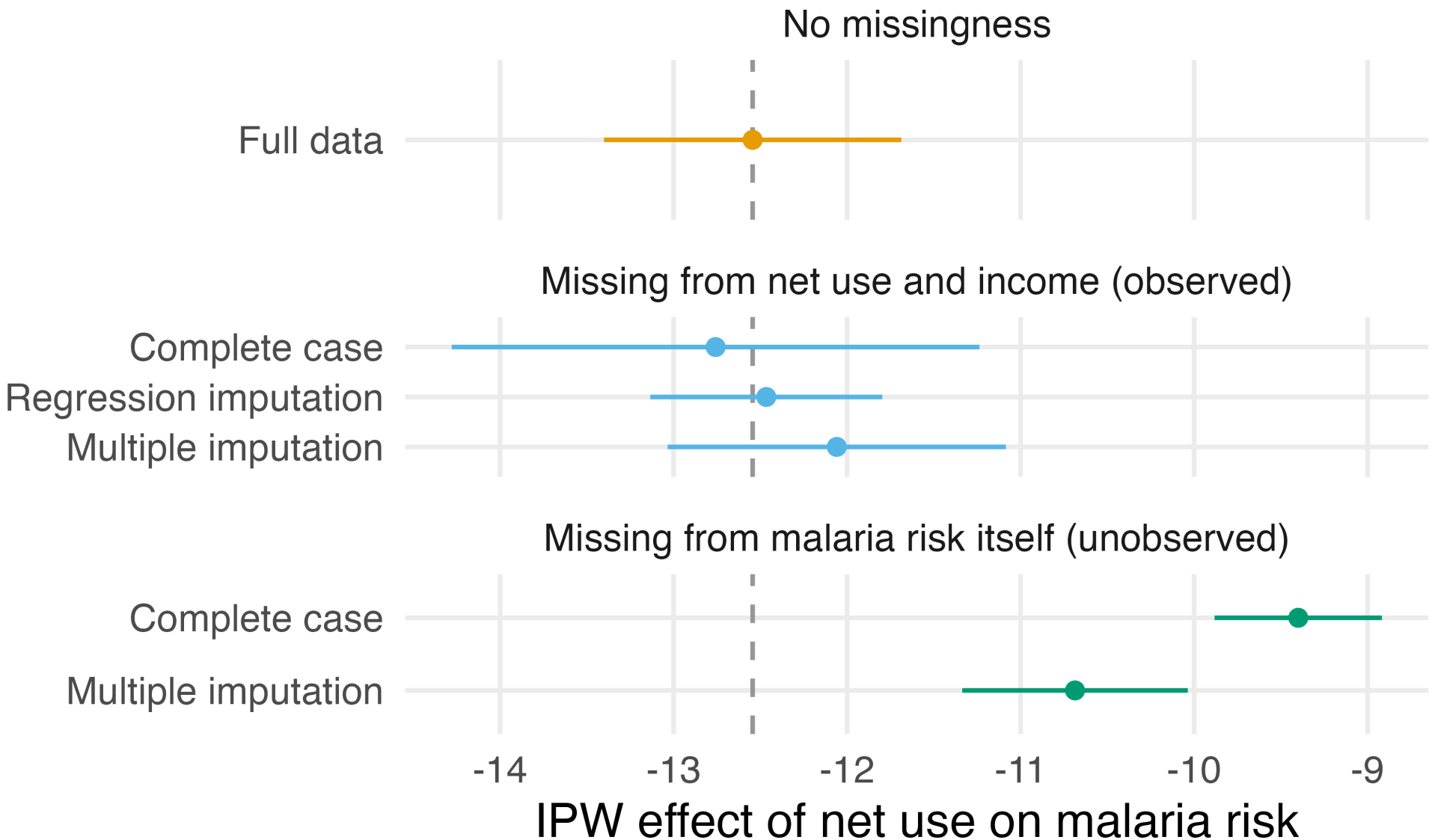
The DAG said as much: there is no way to close the path

What to do instead

- Sensitivity analyses let you probe what you cannot verify; the tools from the tipping point module apply to missingness as well

- When missingness is designed into a study as censoring, weight for it directly with inverse probability of censoring weights, covered in the selection bias module

Comparing the estimates



Comparing the estimates

- Under a recoverable structure, every remedy sits near the benchmark, and multiple imputation reflects the added uncertainty in its standard error

- Under missingness driven by the values themselves, neither estimate is trustworthy: both miss the benchmark, and neither interval includes it

Comparing the estimates

- The data look much the same under both mechanisms; only the structure tells them apart

The real thing is worse

Actual wait times

- In the malaria data we knew the answer; in these data we do not

- The question: the effect of Extra Magic Mornings on the *actual* wait time at 9 AM

Actual wait times

- The 6.2 minutes from the outcome model exercises was the effect on ***posted*** wait times, a different outcome

207 of 354 days are missing the actual wait time, and there is no benchmark to check against

The pipeline for actual wait times

```r
fit_ipw_actual <- function(.data) {
  propensity_model <- glm(
    park_extra_magic_morning ~
      park_ticket_season + park_close + park_temperature_high,
    family = binomial(),
    data = .data
  )
  weighted_data <- propensity_model |>
    augment(type.predict = "response", data = .data) |>
    mutate(
      wts = wt_ate(.fitted, park_extra_magic_morning, exposure_type = "binary")
    )
  ipw_model <- lm(
    wait_minutes_actual_avg ~ park_extra_magic_morning,
    data = weighted_data,
    weights = wts
  )
  ipw(propensity_model, ipw_model)
}
```

Complete cases: 147 days

```
seven_dwarfs_complete <- seven_dwarfs_train_2018 |>
  filter(wait_hour == 9, !is.na(wait_minutes_actual_avg))

fit_ipw_actual(seven_dwarfs_complete) |>
  tidy(conf.int = TRUE)
```

```
# A tibble: 1 × 7
  term  estimate std.error statistic p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl>   <dbl>    <dbl>
1 diff      5.11      8.13     0.629   0.530    -10.8
# ℹ 1 more variable: conf.high <dbl>
```

5.1 minutes (95% CI -10.8, 21.0): an interval four times wider than the one from the posted wait times

Impute the full data

```
seven_dwarfs_to_impute <- seven_dwarfs_train_2018 |>
  mutate(park_ticket_season = as.factor(park_ticket_season)) |>
  select(
    park_date,
    wait_minutes_actual_avg,
    wait_minutes_posted_avg,
    wait_hour,
    park_temperature_high,
    park_close,
    park_ticket_season,
    park_extra_magic_morning
  )
```

Impute the full data

- Every variable in the propensity score and outcome models, the outcome itself, and one strong auxiliary predictor: the posted wait time

- The frame also carries `wait_hour`, a structural auxiliary that lets the imputation borrow across hours, and `park_date`, which the predictor matrix will exclude

Ten imputations

```
predictor_matrix <- make.predictorMatrix(seven_dwarfs_to_impute)
# park_date identifies rows, so it must not predict imputed values
predictor_matrix[, 1] <- 0

imp_disney <- mice(
  seven_dwarfs_to_impute,
  m = 10,
  predictorMatrix = predictor_matrix,
  method = "pmm",
  seed = 1,
  print = FALSE
)
```

Ten imputations against 88.7% missing: well below the rule of thumb

Fit the pipeline in each imputation

```
# mice masks tidyr::complete(), so name the package explicitly
disney_fits <- mice::complete(imp_disney, action = "all") |>
  map(\(.df) fit_ipw_actual(filter(.df, wait_hour == 9)))

disney_estimates <- map_dfr(disney_fits, tidy)

round(disney_estimates$estimate, 2)
```

```
 [1]  6.24 -1.43 11.51 10.61 -1.96  1.36 -2.30  7.20  7.92  2.98
```

Ten completed datasets, ten answers: from -2.3 to 11.5 minutes

Pooling the estimates

```
pool_ipw(disney_fits) |>
  tidy(conf.int = TRUE)
```

```
# A tibble: 1 × 8
  term  estimate std.error statistic    df p.value conf.low
  <chr>    <dbl>     <dbl>     <dbl> <dbl>   <dbl>    <dbl>
1 diff      4.21      7.13     0.591  22.2   0.561    -10.6
# ℹ 1 more variable: conf.high <dbl>
```

4.2 minutes (95% CI -10.6, 19.0): the interval is still nearly as wide as the complete-case one

What we can say

- Ten imputations against 88.7% incompleteness violates the rule of thumb; with more imputations, the pooled estimate varies far less from run to run

- No number of imputations makes the interval informative: 207 of the 354 outcome values are missing

What we can say

- Multiple imputation reflects that in the interval, where deterministic imputation would have hidden it

In the exercises, you will impute the analysis subset directly, with `m` matched to the rule of thumb

Your Turn 1

Open `15-bonus-missingness-exercises.qmd`

Count the days missing the actual wait time and visualize the missingness pattern

Decide which missingness DAG you believe, and what it implies

Your Turn 2

Impute the missing actual wait times with a regression model that keeps the exposure and confounders in their analysis forms

Estimate the effect of an Extra Magic Morning on the actual wait time with the IPW pipeline

Your Turn 3

Multiply impute the hour 9 data with `mice()`, choosing `m` by the rule of thumb

Fit the full IPW pipeline within each imputation and pool with `pool_ipw()`

Finish early? Try the *stretch goals*

10:00